

Match

字符串匹配

本页面将简述字符串匹配问题以及它的解法。

字符串匹配问题

定义

又称模式匹配（pattern matching）。该问题可以概括为「给定字符串 s 和 t ，在 s 中寻找子串 t 」。字符串 t 称为模式串（pattern）。

类型

- 单串匹配：给定一个模式串和一个待匹配串，找出前者在后者中的所有位置。
- 多串匹配：给定多个模式串和一个待匹配串，找出这些模式串在后者中的所有位置。
 - 出现多个待匹配串时，将它们直接连起来便可作为一个待匹配串处理。
 - 可以直接当做单串匹配，但是效率不够高。
- 其他类型：例如匹配一个串的任意后缀，匹配多个串的任意后缀……

暴力做法

简称 BF (Brute Force) 算法。该算法的基本思想是从主串 s 的第一个字符开始和模式串 t 的第一个字符进行比较，若相等，则继续比较二者的后续字符；否则，模式串 t 回退到第一个字符，重新和主串 s 的第二个字符进行比较。如此往复，直到 s 或 t 中所有字符比较完毕。

实现



C++Python

```
1 /*
2  * s: 待匹配的主串
3  * t: 模式串
4  * n: 主串的长度
5  * m: 模式串的长度
6  */
7 std::vector<int> match(char *s, char *t, int n, int m) {
8     std::vector<int> ans;
9     int i, j;
10    for (i = 0; i < n - m + 1; i++) {
11        for (j = 0; j < m; j++) {
12            if (s[i + j] != t[j]) break;
13        }
14        if (j == m) ans.push_back(i);
15    }
16    return ans;
17 }
```

```

1 def match(s, t, n, m):
2     if m < 1:
3         return []
4
5     ans = []
6     for i in range(0, n - m + 1):
7         for j in range(0, m):
8             if s[i + j] != t[j]:
9                 break
10        else:
11            ans.append(i)
12    return ans

```

时间复杂度

设 n 为主串的长度， m 为模式串的长度。默认 $m \leq n$ 。

BF 算法匹配成功时，在最好情况下，只有一趟匹配成功，此趟比较次数为 m ，而其余每趟不成功的匹配都发生在模式串的第一个字符，还需要 m 次比较，总比较次数为 $m(n - m + 1)$ ，故时间复杂度为 $O(mn)$ ；在最坏情况下，匹配成功的趟数为 $n - m + 1$ ，每趟比较次数为 m ，总比较次数为 $m(n - m + 1)$ ，故时间复杂度为 $O(mn)$ 。

BF 算法匹配失败时，在最好情况下，每趟不成功的匹配都发生在模式串的第一个字符，BF 算法要执行 m 次比较，时间复杂度为 $O(m)$ ；在最坏情况下，每趟不成功的匹配都发生在模式串的最后一个字符，BF 算法要执行 m 次比较，时间复杂度为 $O(mn)$ 。

如果模式串有至少两个不同的字符，则 BF 算法的平均时间复杂度为 $O(mn)$ 。但是在 OI 题目中，给出的字符串一般都不是纯随机的。

Hash 的方法

参见：[字符串哈希](#)

KMP 算法

参见：[前缀函数与 KMP 算法](#)

本页面最近更新：2023/11/21 17:04:20，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[iamtwz](#), [abc1763613206](#), [Enter-tainer](#), [f1rstmehul](#), [HeRaNO](#), [lr1d](#), [karman011](#), [ksyx](#), [Menci](#), [NachtgeistW](#), [ouuan](#), [shawlleyw](#), [Xeonacid](#)

本页面的全部内容[在 CC BY-SA 4.0 和 SATA 协议之条款下](#)提供，附加条款亦可能应用